

A Mental Health Project

FEELIO



CSE 318: SYSTEM ANALYSIS AND DESIGN LAB

SUBMITTED BY

Name	ID	Intake
Shadman Shahriar	20245103408	53
Amrin Jahan	20245103009	53
Munasib Maria Oyshi	20245103010	53
Kaniz Fatema	20245103154	53
Md. Sajidul Islam	20245103199	53

SUPERVISED BY:

Shampa Banik

Lecturer,

Department of CSE

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
BANGLADESH UNIVERSITY OF BUSINESS AND TECHNOLOGY (BUBT)**

24th April 2026

Abstract

Mental health issues such as stress, anxiety, and depression are becoming increasingly common among students and professionals. Traditional counseling methods are effective but often lack engaging digital tools that combine real-time feedback with user interaction. Our project “Feelio” offers a simple, stream-lined access to mood tracking, journaling, and companionship that aims to overcome this problem.

List of Figures

Sl	Figure Name	Page
3.2.2	Use case diagram	5
3.2.3	Context-Level Data Flow Diagram	6
5.1.1	E-R Diagram	9
6.1	Project Demonstration: Home Page	12
6.2	Project Demonstration: Daily Check-in	13
6.3	Project Demonstration: Calendar	14
6.4	Project Demonstration: Chat Interface	15
6.5	Project Demonstration: Notification	16

List of Tables

SI	Table Name	Page
1	Database Implementation	10

Table of Contents

Abstract	ii
List of Figures	iii
Chapter: 1 Introduction	1-2
1.1. Problem Specification/Statement	
1.2. Objectives	
1.3. Scope of the Project	
Chapter: 2 Background	3
2.1. Existing System Analysis	
2.2. Supporting Literatures	
2.3. Feasibility Study	
Chapter: 3 System Analysis & Design	4-6
3.1. Technology & Tools	
3.2. Model & Diagram	
3.2.1. Model (Agile)	
3.2.2. Use Case Diagram	
3.2.3. Context Level Diagram	
Chapter: 4 System Specification	7-8
4.1. System Specification	
4.1.1 Functional Requirements	
4.1.2 Non-Functional Requirements	
4.2. Tools and Technology	
4.2.1 Programming languages	
4.2.2 Frameworks	
4.2.3 Database systems	
4.2.4 Other tools	
Chapter: 5 Implementation	9-11
5.1. Database Design	
5.2. System Overview	
5.3. Description of front-end and back-end components	
Chapter: 6 Project Demonstration	12-16
6.1. Home Page	
6.2. Daily Check-in	
6.3. Dashboard and All Modules	
6.4. Admin Dashboard and All Modules	
Chapter: 7 Conclusion	17-19
7.1. Conclusion	
7.2. Limitation	
7.3. Future Works	

Chapter 1

INTRODUCTION

1.1. Problem Specification

Our project “Feelio” offers a simple, stream-lined access to mood tracking, journaling, and companionship. We plan to build “Feelio” because most of the current solutions felt short in one way or another. Our aim is to build a project that -

1. Does not involve a complex learning curve,
2. Helps its users to track and visualize their emotional wellbeing in different sections (weekly, monthly, yearly) and statistical ways,
3. Export a document consisting of user data that can be shared with their therapists (if any) for necessary medical attention,
4. Provides companionship using an AI-powered chatbot interface where users can express how they feel.

1.2. Objectives

The primary objectives of “Feelio” are -

1. To design an interactive Java Springboot application to help users to track and visualize their emotional wellbeing.
2. To provide emotional support through AI-powered chatbot.
3. To store and analyse user data in a relational database and suggest AI feedback in the chatbot interface.
4. To keep the experience immersive using animations and visuals.

1.3. Scope

The scope of the Feelio project encompasses the development of a streamlined mental health application designed to provide users with tools for emotional tracking and support. The project's boundaries and deliverables are defined by the following areas:

Functional Scope

The application is designed to provide several core functionalities for its users:

- **Daily Mood Tracking:** A calendar-based interface where users can record their moods using dates and markers.
- **Journaling:** A feature allowing users to generate and store text-based journal entries.
- **AI-Powered Companionship:** An interactive chat interface that utilizes the Gemini API to provide mindful conversations and emotional support.

- **Data Visualization and Analysis:** Tools to help users visualize their emotional well-being over various timeframes (weekly, monthly, yearly) using statistical methods.
- **Data Portability:** The ability to export user data into a CSV document that can be shared with therapists for professional medical attention.

Technical Scope

The project involves specific technical implementations and requirements:

- **Platform:** Development of an interactive Java Springboot application framework.
- **Data Management:** Utilizing a relational database (SQLite3) for local storage of user information and activity.
- **Integration:** Fetching real-time responses from a Gemini endpoint and using a Webview Engine.
- **Operational Flexibility:** The system is designed for both online and offline use, as data is stored locally; however, an internet connection is required specifically for the AI chatbot.
- **Hardware Requirements:** The application requires at least 4 GB of RAM to run.

User Experience and Design Scope

- **Target Audience:** The application specifically targets students and professionals facing stress, anxiety, or depression.
- **Ease of Use:** A primary goal is to ensure a simple learning curve compared to existing solutions.
- **Immersive Design:** The scope includes the use of animations and visuals to keep the user experience engaging.
- **Internationalization:** The software is designed to support different languages through separated language modules (.lang files).

Out of Scope / Constraints

- **Authentication:** No complex authentication is required because the software operates solely on the end-user's local hardware.
- **Medical Advice:** While it provides AI feedback, the system is intended as a support tool, and data is meant to be exported to a professional therapist for actual medical attention.

Chapter 2

BACKGROUND

2.1. Existing System Analysis

Headspace: Similar to our implementation, Headspace also features an AI chatbot interface that determines the best healing exercises based on the user's performance. However, Headspace is a subscription-based app where most of its essential features are paywalled.

Daylio: This app is the closest implementation to our system. However, unlike our implementation, Daylio is online-based and doesn't feature an AI chatbot interface.

Moodfit: This app features a set of tools to manage one's emotional well-being. But their technical UI might be difficult to navigate for certain users.

2.2. Supporting Literatures

The supporting literature for the Feelio project consists of analyses of existing mental health and mood-tracking applications, which are already cited in the section above.

2.3. Feasibility Study

- **Technical Feasibility:** The system requires a Java application framework, SQLite3 database, Gemini API, and Webview Engine. These technologies are widely available. This makes the system technically feasible. However, lack of technical skills from new engineers might result in prolonged delays and missing deadlines.
- **Economic Feasibility:** Initial costs include system development, API charges, and maintenance. However, our free tier will increase user adaptability, resulting in more people using the system. If benefits outweigh costs, the system will be economically feasible. But if the system fails to reach their target demographic, the project may fail.
- **Operational Feasibility:** Since the user data is stored locally, the system can be used in both online and offline. The chatbot interface falls back gracefully by notifying the user that there is no Internet connection. So, the system is operationally feasible. However, if an user exclusively wants to use the chatbot interface, an Internet connection is required.

Chapter 3

SYSTEM ANALYSIS & DESIGN

3.1. Technology & Tools

- **Application Framework:** The project is designed as an interactive Java application.
- **Database Management:** It utilizes SQLite3, a relational database, to store user activity and information locally.
- **AI Integration:** The system fetches responses from the Gemini API (specifically the Gemini endpoint) to power its interactive chatbot interface.
- **Rendering Engine:** A Webview Engine is integrated into the technical framework.

3.2. Model & Diagram

3.2.1. Model (Agile): We are using the Agile methodology for our project among SDLC, Agile or OOSAD. We chose Agile because -

Agile is a flexible framework, allowing rapid changes and providing incremental updates.

We can add new features while we solidify the core features and postpone the high risk features until they are ready to be implemented.

Since Agile is a user-centric development framework, we can divide the work in short-cycles, conduct daily meetings, test software and collaborate as a team.

3.2.2. Use Case Diagram

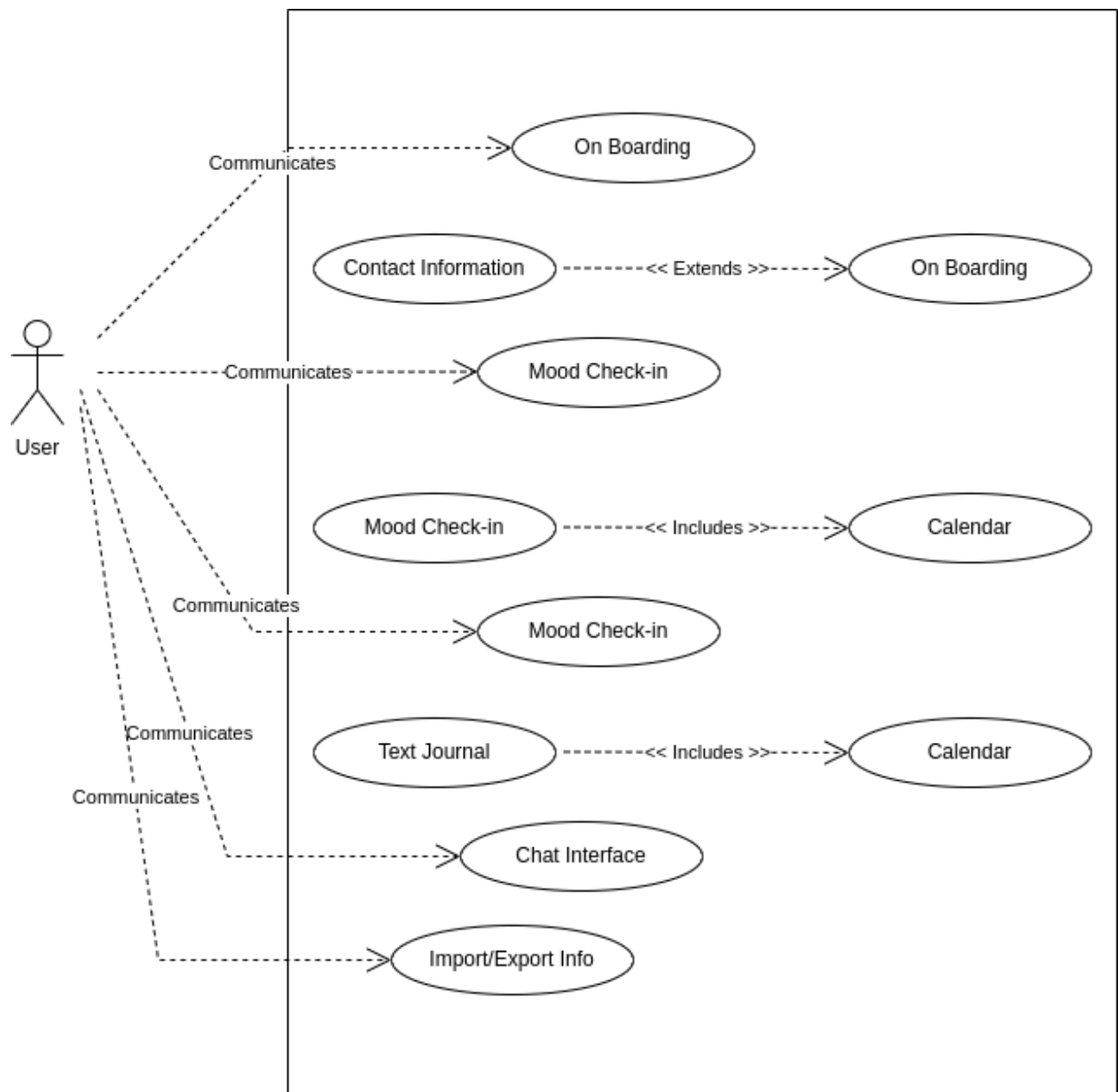


Fig 3.2.2: Use case diagram of our proposed project.

3.2.3. Context Level Diagram

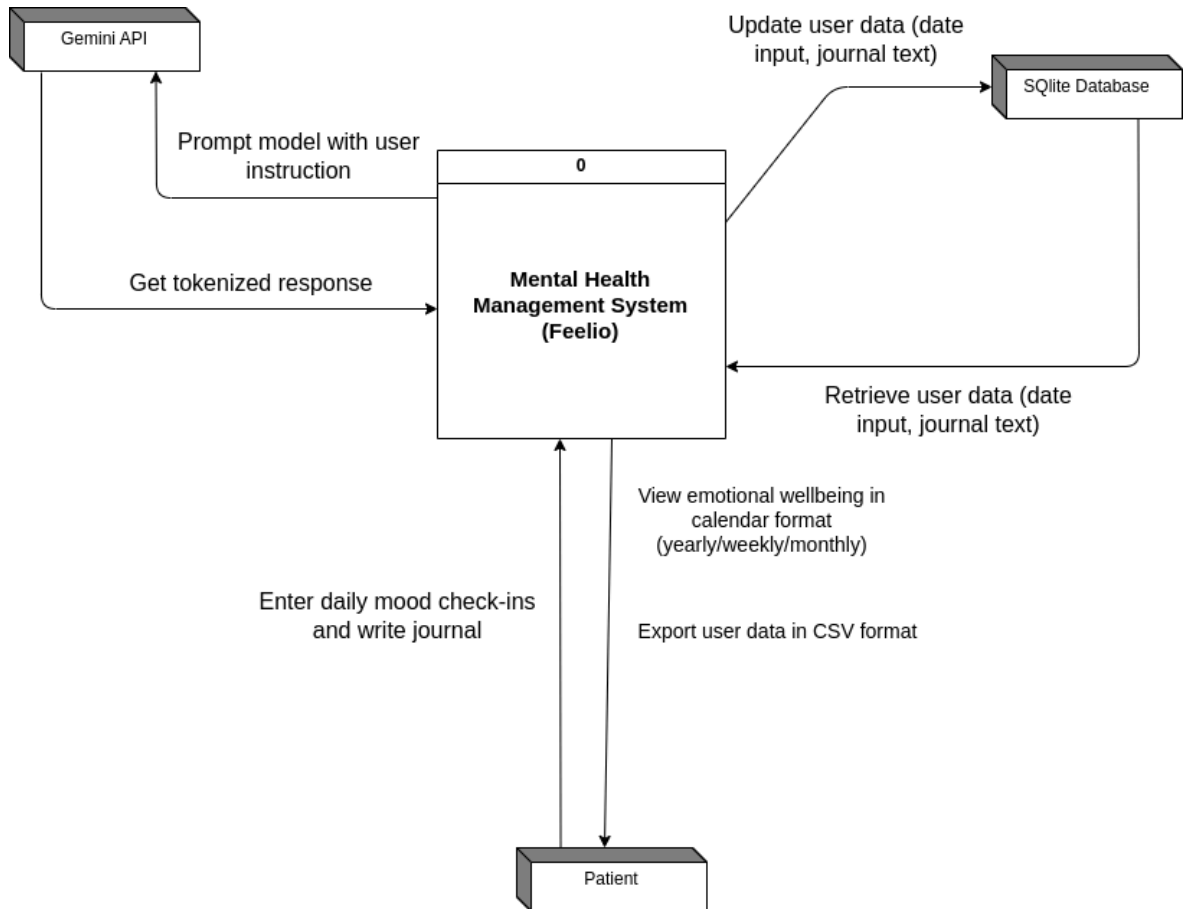


Fig 3.2.3: Context-Level Data Flow Diagram of the proposed system.

Chapter 4

SYSTEM SPECIFICATION

4.1. System Specification

4.1.1. Functional Requirements

REQ-1: Date/Time component

REQ-2: Textarea, text field, buttons

REQ-3: Message interface

REQ-4: Searchbox

4.1.2. Other Non-functional Requirements

- **Performance Requirements**

- At least 4 GB of RAM is required to develop this project.
- Having graphics memory is optional.

- **Safety Requirements**

- Gemini API should be prompted to refrain from its output being suggestive and explicit. A model protocol can be set up before initializing the API.

- **Security Requirements**

- No authentication is required since the software solely operates in the end-user's hardware.

- **Software Quality Attributes**

- Each cumulative update should be tested and verified by an authorized developer or tester.
- Each release should be thoroughly tested by a system tester, preferably using an automated service.
- End-users can act as beta testers and suggest feedback through the in-app feedback mechanism.
- Crucial updates and bug fixes can be provided over the air or using the in-app updater.

- **Other Requirements**

- Database systems are subject to change but a relational model is preferred.
- Software's language file is separated into language modules (.lang files) to ensure internationalization.
- The software may use open-source codes usually licensed under MIT, GNU GPL 3, or Creative Commons CC rev 4.

4.2. Tools and Technology

4.2.1 Programming Languages

The primary programming language used for the development of the Feelio project is Java. The project is specifically designed as an interactive Java application.

4.2.2 Frameworks

It used the Springboot Framework to create a web server.

4.2.3 Database Systems

SQLite3 is used as the relational database for storing user activity and information locally

4.2.4 Other Tools

Feelio uses GitHub for version management.

Chapter 5

IMPLEMENTATION

5.1. Database Design

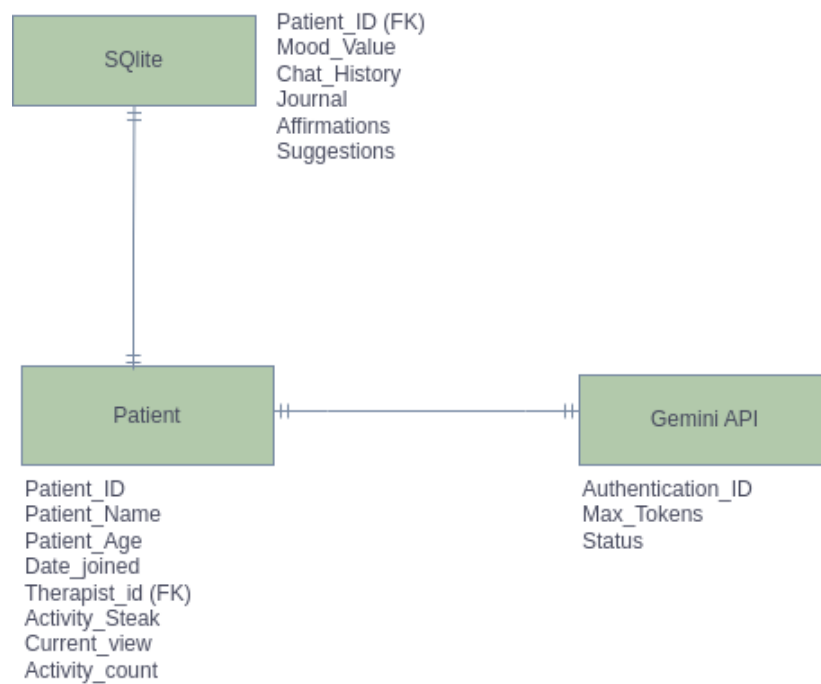


Fig 5.1.1: E-R Diagram of the proposed system.

Table Name	Attributes	Description
Patient	● Patient_ID (PK), ● Patient_Name, ● Patient_Age, ● Date_joined, ● Therapist_id (FK), ● Activity_Steak, ● Current_view, ● Activity_count	Stores primary user profile information and app usage statistics.
SQLite (Records)	● Patient_ID (FK), ● Mood_Value, ● Chat_History, ● Journal, ● Affirmations, ● Suggestions	Stores the actual content of the user's mental health tracking, including moods, text journals, and AI interactions.
Gemini API (Config)	● Authentication_ID, ● Max_Tokens, ● Status	Manages the configuration and status of the external AI. interface for the specific user.

5.2. System Overview

The Feelio system provides a multi-faceted approach to mental health support, which can be categorized into several high-level solution types based on its functionality and technical architecture:

- **Emotional Tracking and Visualization Solution:** The system allows users to track their emotional well-being using a Daily Mood Tracking calendar interface. It transforms this data into visual formats—including weekly, monthly, and yearly views—using statistical methods to help users understand their emotional patterns over time.
- **AI-Powered Companionship and Support:** By integrating the Gemini API, the system provides a chat interface that offers mindful conversations and emotional support. This serves as an interactive companion where users can express their feelings and receive AI-generated feedback.
- **Local Data Management and Privacy Solution:** Unlike many online-only applications, Feelio is designed for operational flexibility, storing user activity and journaling data locally in an SQLite3 relational database. This allows the system to function offline for mood tracking and journaling, though an internet connection is required for the AI chatbot.
- **Professional Integration Solution:** The system bridges the gap between digital self-care and professional help by allowing users to export their data in CSV format. This document can then be shared with therapists for professional medical attention.

5.3. Description of Front-end and Back-end Components

Front-end: The interface is built around several key functional elements as defined in the system's requirements:

- **Daily Mood Tracking Calendar:** A visual interface consisting of dates and markers where users can input and view their emotional state.
- **Journaling Interface:** A space for a collection of user-generated text content. It includes UI elements such as textareas, text fields, and buttons.
- **AI Chat Interface:** A message interface that displays tokenized responses fetched from the Gemini API.
- **Navigation and Search:** The UI includes a search box for finding information and a date/time component for logging entries.
- **Internationalization:** The software supports different languages by using separated .lang modules, allowing the interface to be adapted for global users.

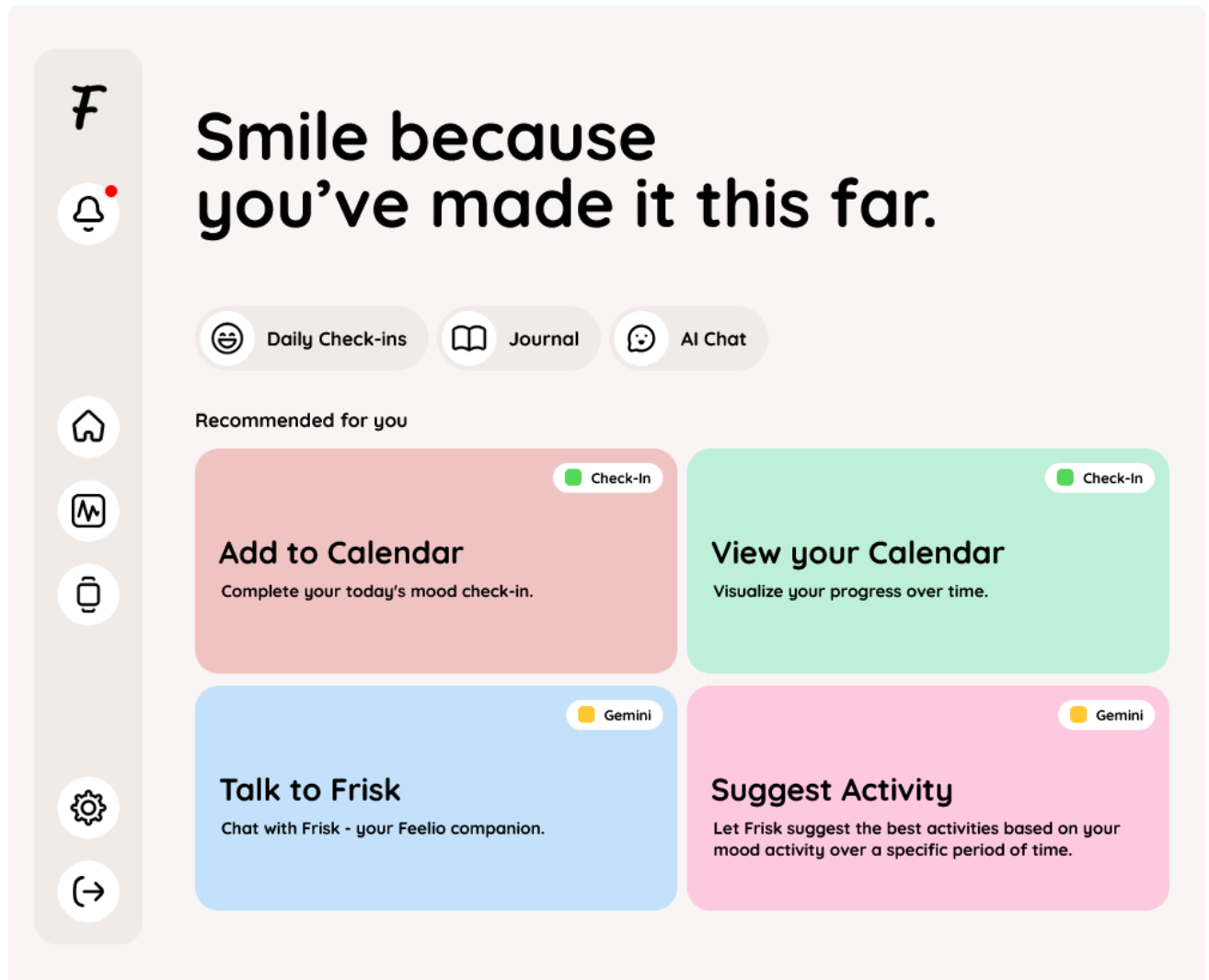
Back-end: The project utilizes SQLite3 as its primary data storage solution.

- **Type and Relationship:** It is a relational database. In the system's Entity-Relationship (E-R) diagram, SQLite is treated as a fundamental entity with a 1-to-1 relationship with the Patient.
- **Stored Information:** The database stores user activity and basic information, including:
 - Patient ID (*Foreign Key*)
 - Mood Values
 - Journal entries
 - Chat History
 - Affirmations and Suggestions
- **Data Portability:** Logic is included to export these database records into a CSV format for external use by therapists.

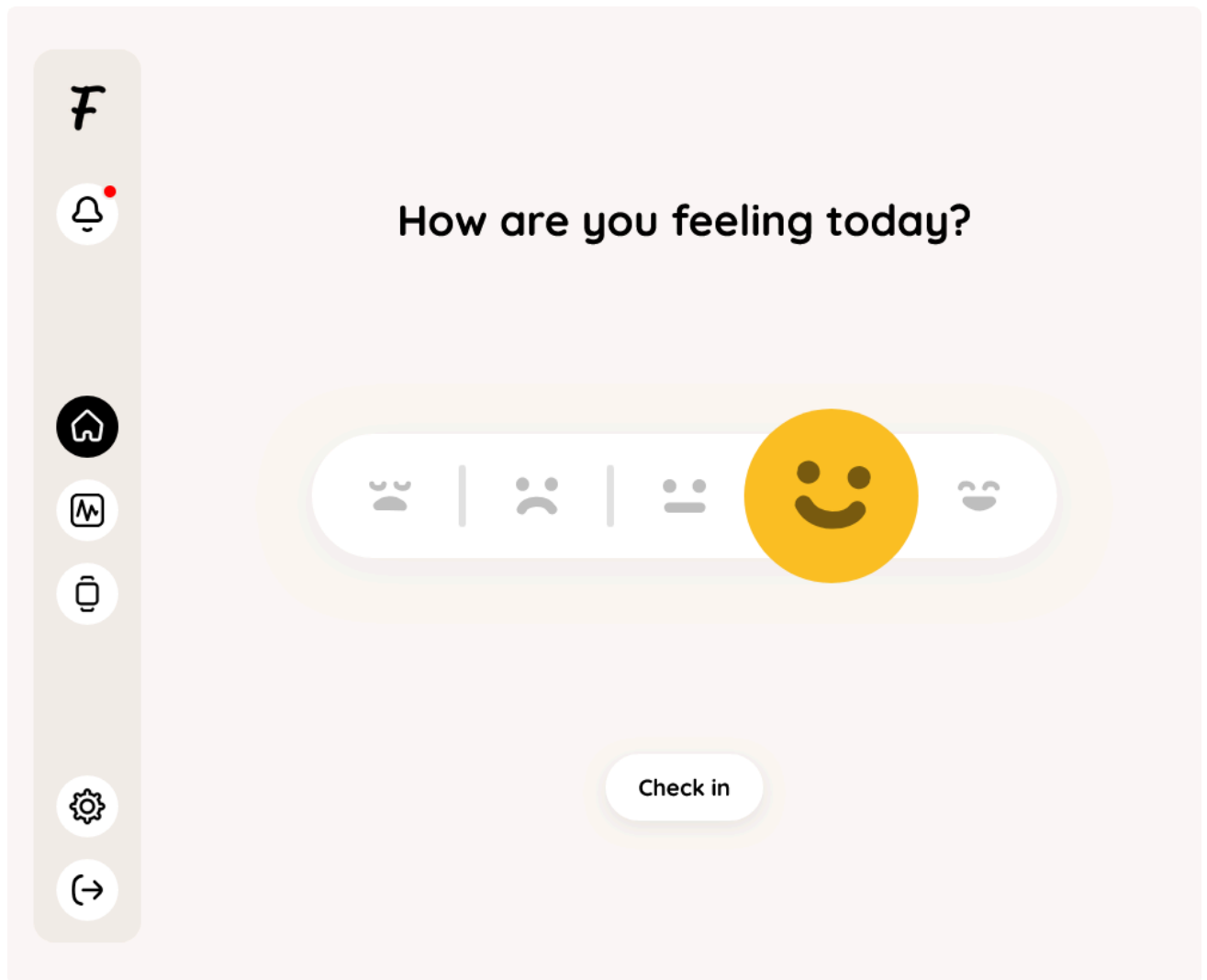
Chapter 6

PROJECT DEMONSTRATION

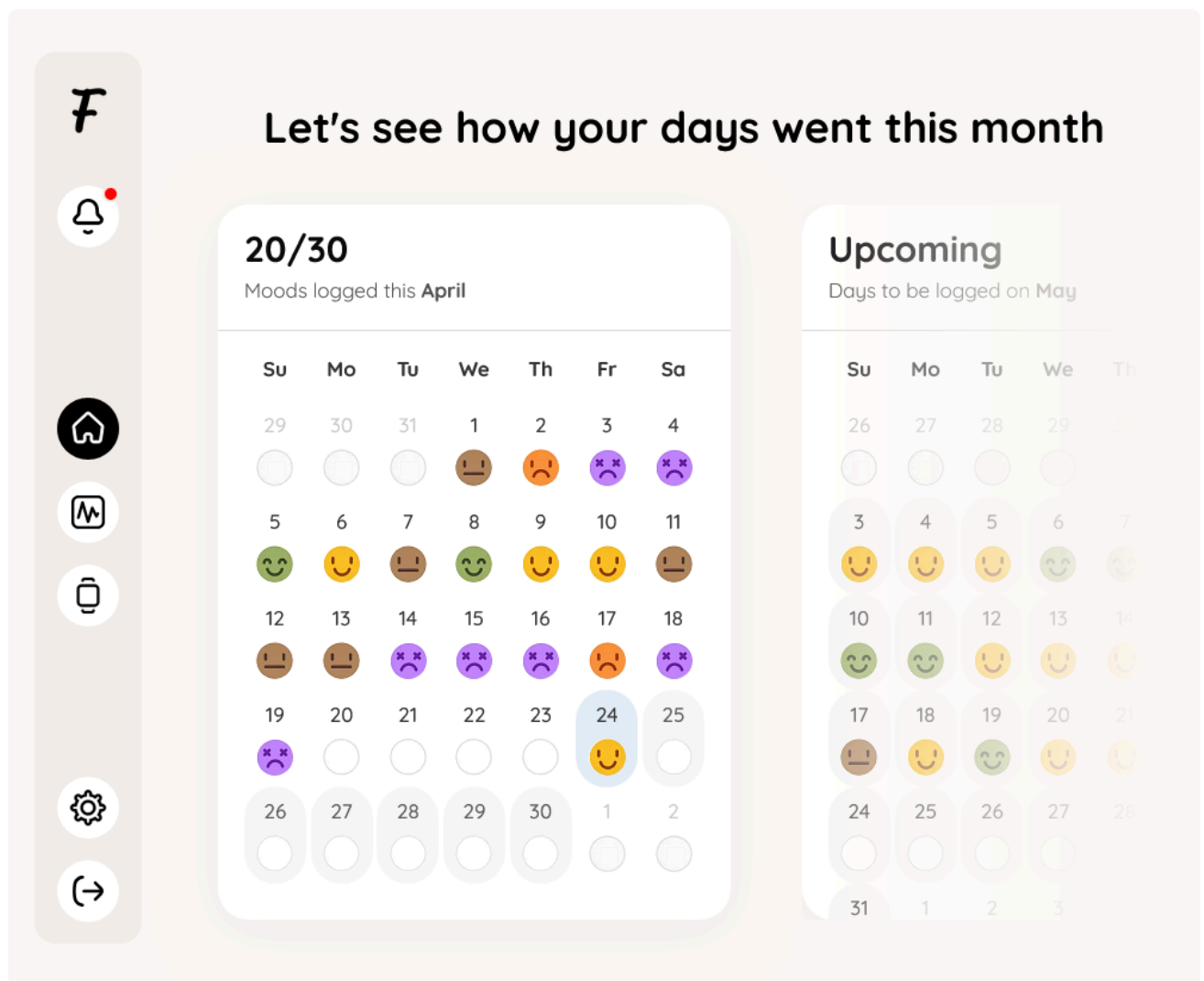
6.1. Home Page



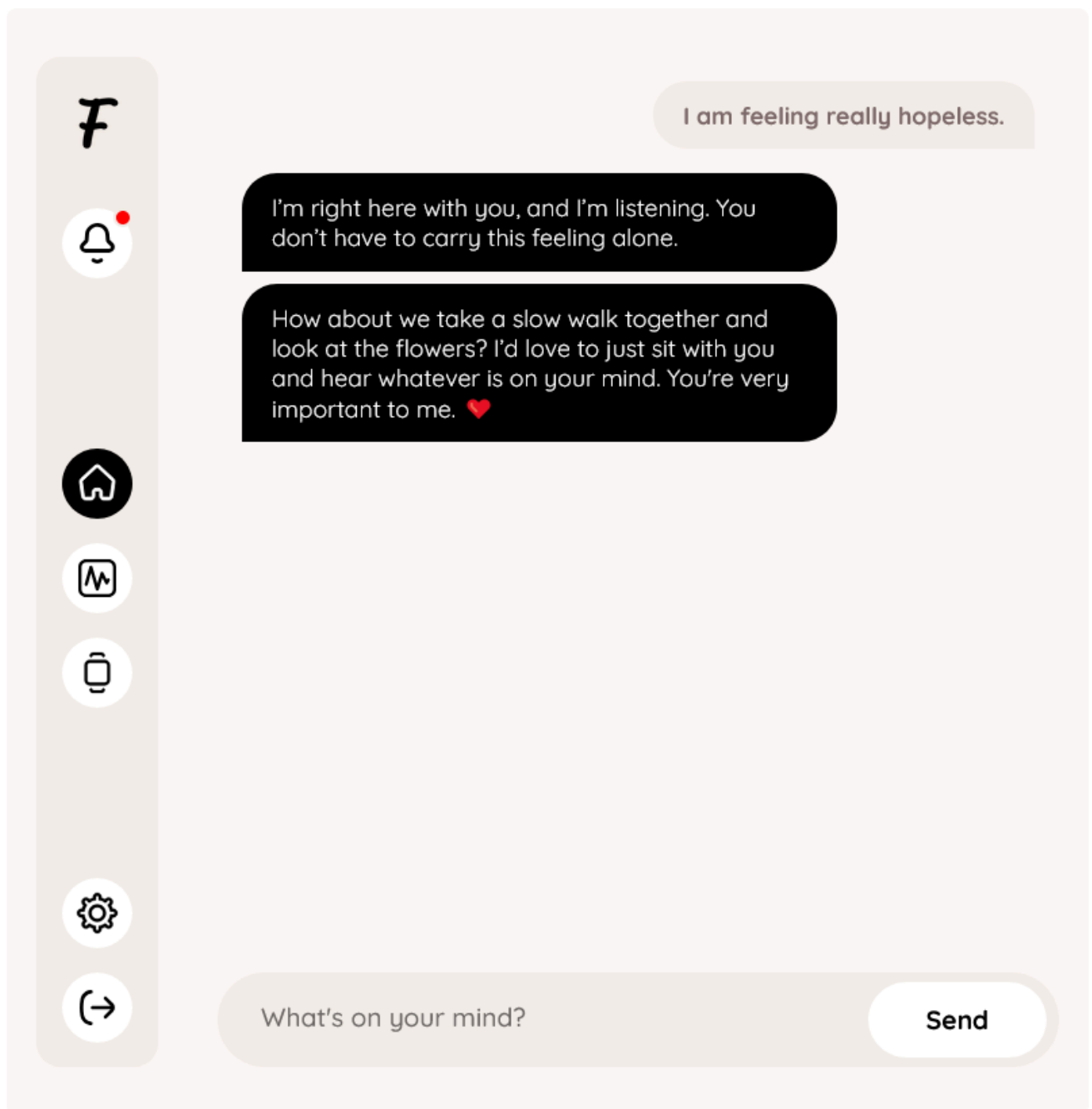
6.2. Daily Check-in



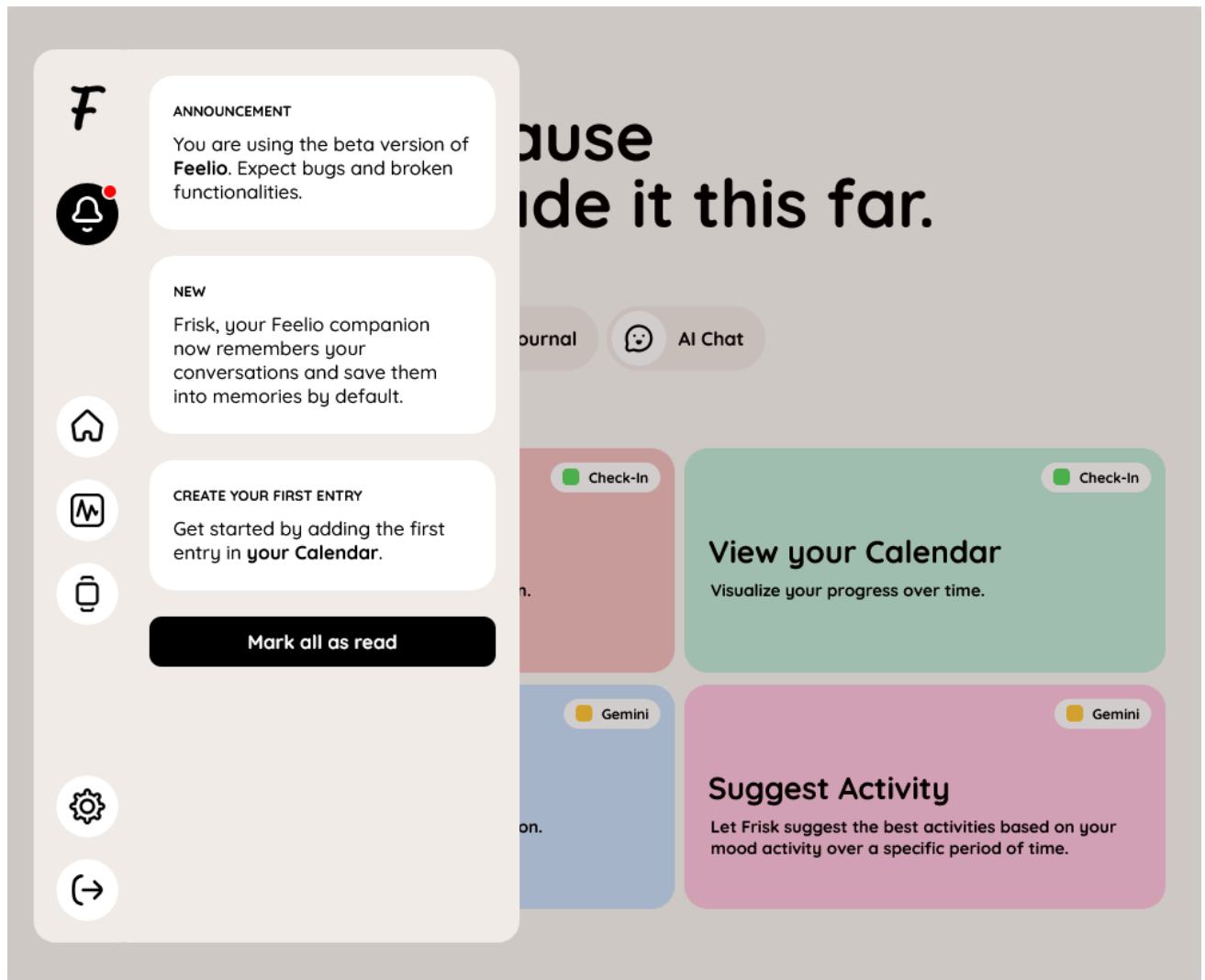
6.3. Calendar



6.4. Chat Interface



6.5. Notification



Chapter 7

CONCLUSION

7.1. Conclusion

In conclusion, the Feelio project addresses a critical gap in the digital mental health space by providing a streamlined, interactive tool for students and professionals facing stress and anxiety.

The analysis of existing systems revealed that current solutions often fall short due to subscription-based paywalls (Headspace), a lack of AI-powered companionship (Daylio), or overly complex user interfaces (Moodfit). Feelio addresses these issues by offering a simple learning curve, an offline-first local database for privacy and flexibility, and AI-powered emotional support via the Gemini API.

Through the development process, several key insights were gained regarding project management and technical implementation:

- **Methodology:** The Agile framework was found to be the most effective approach, as its flexibility allows for rapid changes, incremental updates, and the ability to postpone high-risk features until core functionalities are solidified.
- **Feasibility:** The project was determined to be technically and operationally feasible by leveraging widely available technologies like Java, SQLite3, and the Gemini API. However, it was noted that a high level of technical skill is required to avoid delays.
- **User Privacy:** Storing data locally on the end-user's hardware was identified as a core strength, as it removes the need for complex authentication and ensures the app remains functional in offline environments.

7.2 Limitations

Technical and Resource Limitations

- **Engineering Skill Gap:** A significant risk noted is that a lack of technical skills among new engineers could lead to prolonged delays and missed deadlines during development.
- **Hardware Requirements:** The system has a specific hardware threshold, requiring at least 4 GB of RAM to function, which may limit accessibility for users with older or lower-spec devices.
- **Database Volatility:** While a relational model is preferred, the sources indicate that database systems are subject to change, suggesting the current architecture may need future adjustments.

Operational and Functional Limitations

- **Internet Dependency for AI:** While the system is designed for "operational flexibility" with local storage, the chatbot interface requires an active internet connection to fetch responses from the Gemini endpoint.
- **AI Output Safety:** There is an inherent need for strict safety protocols. The Gemini API must be specifically prompted and configured with a model protocol to prevent it from generating suggestive or explicit content.
- **Feature Risk Levels:** Not all features carry the same level of stability. While mood tracking is low-risk, the Chat Interface is classified as a HIGH risk feature with a high cost penalty, indicating it is the most complex and potentially unstable part of the system.

Economic and Domain Limitations

- **Market Reach:** The project's economic feasibility is tied to its user base. If the application fails to reach its target demographic (students and professionals), the project may not be sustainable.
- **API Costs:** Initial economic analysis highlights that API charges are a recurring cost that must be balanced against the project's free-tier model.
- **Digital Tool vs. Professional Care:** Within the mental health domain, the project acknowledges a limitation in that digital tools often lack the engagement of traditional counseling. Consequently, the system is designed to export data to a therapist rather than replacing professional medical attention.

7.3 Future Works

1. Refinement of High-Risk Features

- Advanced AI Chat Integration:** Currently, the Chat Interface is classified as a low priority and high risk feature. Future work could focus on solidifying this core feature, improving prompt engineering for more nuanced "mindful conversations," and further developing the model protocol to ensure the AI remains safe and non-suggestive.
- Enhanced Data Analysis:** While the current system provides basic visualization (weekly, monthly, yearly), future versions could implement more advanced statistical methods or predictive analytics to help users identify long-term mental health trends.

2. System Infrastructure and Maintenance

- Automated Update System:** The proposal mentions providing crucial updates and bug fixes through an in-app updater or "over the air". Developing this infrastructure would be a practical technical goal for the next SDP.
- Database Optimization:** Since "database systems are subject to change," future work could involve optimizing the SQLite3 relational model or exploring alternative local storage solutions as the user data grows in complexity.

3. Expansion of Accessibility and Reach

- a. **Full-Scale Internationalization:** The software is designed to support language modules (.lang files). A future objective would be the actual translation and implementation of these modules for a wider global audience.
- b. **Refinement of the User Interface:** Based on user reactions observed during the prototype phase (Activity F), future work could involve iterative UI improvements to maintain the goal of a simple learning curve compared to competitors like Moodfit.

4. Testing and Community Engagement

- a. **Integration of Feedback Loops:** Developing the in-app feedback mechanism more fully would allow end-users to act as beta testers, providing direct data for future refinements.
- b. **A/B Testing:** The project schedule plans for A/B testing once enough data is available during early adoption. Implementing these tests would be a key task for the next development cycle to optimize user engagement.

5. Funding and Scaling

The final phase of the current plan involves presenting at tech conventions and raising funds to further facilitate development. Successfully securing this support would allow the team to move beyond a prototype and into a more robust production environment.
